

RED TEAM LEARNING PATH

MODULE 16

LINUX PRIVILEGE ESCALATION

Manual enumeration, sudo/SUID/capabilities, scheduled jobs, credentials, containers, automation & remediation

Authorized-lab only Semua privilege-escalation hands-on dilakukan pada container training yang sengaja vulnerable dan terisolasi. Jangan menyalin teknik ini ke host, kantor, kampus, cloud VM, atau sistem lain tanpa scope dan izin tertulis.

Omarchy / Arch Linux Edition

Project: VaxXinE/red-teaming | Version 1.0 | 05 Oktober 2026

Field	Value
Primary OS	Omarchy / Arch Linux
Prerequisite	Modules 00-15
Core tools	sudo, find, getcap, systemctl, journalctl, ps, ss, pspy, LinPEAS
Hands-on	Disposable Docker lab; no host mounts; no network
Guided labs	16 labs + capstone
Next module	17 - Windows Privilege Escalation

Module map

Phase	What you will do
Manual discovery	Identity, OS, groups, sudo, SUID/SGID, capabilities, services, jobs, PATH, secrets.
Hypothesis	Connect privilege-bearing executor to attacker-controlled input or weak policy.
Controlled proof	Read one root-only flag or observe root-executed artifact; avoid persistence.
Validate & compare	Reset lab, retest, then compare manual findings with pspy/LinPEAS.
Report & remediate	Explain primitive, telemetry, cleanup and the configuration change that removes it.

Lab containment Training container uses no host bind mount, no Docker socket and `--network none`. Resource limits are applied. The intentionally vulnerable privilege primitives exist only inside this disposable container.

1. Learning Objectives & Safety Boundary

[] Membangun manual privilege-escalation checklist sebelum menjalankan automation.

[] Memahami identity, effective privilege, groups, sudo policy, file ownership, SUID/SGID, Linux capabilities, services, timers, cron, mounts, PATH, dan credential exposure.

[] Membedakan evidence of misconfiguration dari exploitability; tidak semua SUID/capability adalah vulnerability.

[] Membuktikan impact dengan minimum sufficient proof pada disposable lab, bukan langsung membuat persistent root shell.

[] Menggunakan pspy/LinPEAS sebagai pembanding setelah manual enumeration, bukan sebagai pengganti reasoning.

[] Mengenali Docker socket/group sebagai root-equivalent boundary dan mempertahankan default Omarchy yang lebih aman.

[] Mendokumentasikan attack path, telemetry, cleanup, dan remediation yang actionable.

Success condition Tujuan module bukan "mendapat root secepat mungkin". Tujuannya: enumerate -> form hypothesis -> validate one path -> collect minimal evidence -> explain root cause -> remediate -> retest.

Privilege escalation mental model

```

Low-privilege foothold
|
v
Identity + OS + groups + environment
|
v
Privilege boundaries / trusted executors
|
v
Writable input? weak policy? leaked credential?
|
v
Controlled proof of elevated access
|
v
Evidence + cleanup + remediation
  
```

2. Taxonomy: Where Linux PrivEsc Usually Comes From

Vector	Question to ask	Typical evidence
sudo	Apa command yang boleh dijalankan sebagai user lain/root?	`sudo -l`, sudoers rule, command semantics.
SUID/SGID	Binary apa yang berjalan dengan owner/group privilege?	Mode bit, owner, binary behavior.
Capabilities	Privilege root apa yang dipisah ke executable?	`getcap -r /`, capability set.
Scheduled execution	Apa yang root jalankan periodik dan apakah input/script writable?	cron, systemd timer, pspy, ownership.
Services	Service root membaca config/binary/path dari mana?	unit file, ExecStart, writable file/path.
Credentials	Ada secret reusable di config/history/process args?	config permissions, env, command line.
Containers	Apakah user punya akses ke privileged runtime/socket?	groups, `/var/run/docker.sock`, mounts.
Kernel	Apakah local kernel vulnerable dan exploit applicable?	kernel/build, vendor advisory, mitigations.

Prioritize configuration before kernel exploits Misconfiguration biasanya lebih deterministic dan lebih mudah diremediate. Kernel exploit adalah last-resort path: higher crash risk, architecture/build sensitive, dan bisa mengganggu engagement.

3. Manual Enumeration: First Five Minutes

Mulai dari facts. Jangan menjalankan enumerator besar sebelum lo tahu user, host, kernel, distro, network context, dan basic privilege boundary.

```

whoami
id
uname -a
cat /etc/os-release
hostname
ip -brief addr
ip route
ss -lntup 2>/dev/null | head -n 40
ps auxww | head -n 30
  
```

Environment & filesystem quick checks

```

printf '%s\n' "$PATH" | tr ':' ' '
mount | column -t | head -n 40
findmnt -o TARGET,SOURCE,FSTYPE,OPTIONS | head -n 40
ls -ld /tmp /var/tmp /opt /usr/local/bin 2>/dev/null
  
```

Evidence discipline Simpan output penting ke workspace. Jangan dumping semua `/proc`, home directories, browser data, SSH keys, atau secrets "just in case". Collect only what supports an authorized hypothesis.

4. Identity, Groups & sudo

`sudo` adalah elevation-control mechanism. `sudo -l` menunjukkan rule yang berlaku untuk current identity dan harus menjadi early check pada Linux target.

```
id
groups
sudo -l
```

What to reason about

- Apakah command allowed sebagai root atau user privileged lain?
- Apakah rule membutuhkan password? NOPASSWD meningkatkan exploitability tetapi bukan satu-satunya issue.
- Apakah command menerima user-controlled file/path/config/plugin/editor/pager?
- Apakah wildcard atau argument restrictions benar-benar membatasi behavior?
- Apakah environment variable dipertahankan oleh sudoers (`env\_keep`, `SETENV`)?

Do not blindly copy GTFOBins GTFOBins adalah reference untuk legitimate Unix binary behaviors yang dapat menjadi security issue pada SUID/sudo/capability context. Pakai hanya setelah lo membuktikan binary dan permission memang in-scope; pahami command sebelum menjalankan.

5. File Permissions, SUID & SGID

SUID executable berjalan dengan effective UID owner; SGID dapat memberi effective group identity. Bit ini bukan vulnerability secara otomatis - risk bergantung pada behavior executable dan input yang dapat dikontrol user.

```
find / -xdev -perm -4000 -type f 2>/dev/null
find / -xdev -perm -2000 -type f 2>/dev/null
stat -c '%A %a %U:%G %n' /path/to/binary
```

Analysis questions

- Apakah binary standard distro atau custom application?
- Apakah dia memanggil external program, membaca user-controlled path/config, atau menggunakan relative path?
- Apakah privilege diturunkan (drop privileges) sebelum operasi berbahaya?
- Apakah owner/mode sesuai dengan vendor package?

Minimum proof Di local lab kita membuktikan arbitrary privileged file read, bukan membuat permanent SUID shell. Itu sudah cukup untuk menunjukkan confidentiality impact dan escalation potential.

6. Linux Capabilities

Linux capabilities membagi privilege superuser menjadi unit terpisah. Contoh: `CAP\_DAC\_READ\_SEARCH` dapat melewati permission check tertentu untuk membaca/search filesystem; `CAP\_DAC\_OVERRIDE` lebih luas. File capability perlu dianalisis terhadap fungsi executable.

```
getcap -r / 2>/dev/null
getcap /path/to/binary
capsh --print 2>/dev/null | head -n 40
```

Capability	Kenapa sensitif
CAP_DAC_READ_SEARCH	Dapat melewati read/search permission checks.
CAP_DAC_OVERRIDE	Dapat melewati banyak discretionary access checks.
CAP_SETUID / SETGID	Dapat mengubah process identity.
CAP_SYS_ADMIN	Sangat luas; sering disebut capability "catch-all" dan perlu scrutiny tinggi.
CAP_SYS_PTRACE	Dapat membuka process-inspection/credential exposure paths tergantung scope.

Context matters `getcap` menemukan capability, bukan finding. Finding baru valid setelah lo menghubungkan capability + executable behavior + attacker-controlled input + impact.

7. Services, systemd Units, Timers & Writable Paths

```
systemctl --type=service --state=running --no-pager
systemctl list-timers --all --no-pager
systemctl cat example.service
systemctl show example.service -p User -p Group -p ExecStart -p EnvironmentFiles
```

Cari trust transition: service berjalan root, tetapi binary/script/config/EnvironmentFile berada pada path yang bisa ditulis low-privilege user. Juga periksa parent directory permissions dan symlink resolution.

```
namei -l /opt/example/bin/start.sh
stat -c '%A %U:%G %n' /opt/example/bin/start.sh
find /etc/systemd /usr/lib/systemd -type f -writable 2>/dev/null
```

Do not modify host units for practice Hands-on scheduled execution dilakukan di disposable container. Jangan membuat unit/cron root yang vulnerable di Omarchy host lo sendiri.

8. Cron, Periodic Jobs & Process Observation

```
cat /etc/crontab 2>/dev/null
ls -la /etc/cron.d /etc/cron.hourly /etc/cron.daily 2>/dev/null
ps auxww --forest | head -n 60
```

Sebagian privileged jobs sulit terlihat dari static files karena user tidak bisa membaca root crontab. `pspy` membantu mengamati short-lived processes tanpa root dengan memonitor procfs dan filesystem events.

Optional pspy workflow

- Ambil binary dari official DominicBreuker/pspy release atau build source sendiri.
- Verifikasi provenance; jangan `curl | sh` tool enumeration dari random mirror.
- Jalankan pada authorized target dan simpan output yang relevan saja.

```
./pspy64 -p -f=false
```

Telemetry is a two-way lesson pspy juga mendemonstrasikan kenapa secret tidak boleh dilewatkan sebagai command-line argument: user lain dapat melihat process command line pada banyak konfigurasi.

9. PATH, Environment & Command Resolution

Privileged process yang mengeksekusi command tanpa absolute path dapat mempercayai `PATH`. Risk meningkat ketika user dapat menulis ke directory yang dicari lebih awal atau environment dipertahankan melintasi privilege boundary.

```
printf '%s\n' "$PATH" | tr ':' '\n'
command -v tar
type -a tar
find / -type d -writable 2>/dev/null | head -n 40
```

Avoid noisy global writable scans first `find / -writable` bisa noisy dan mahal. Mulai dari directories yang terlihat pada PATH, service config, application tree, atau scheduled job. Narrow the hypothesis.

Shared library / loader concepts

Dynamic loader variables seperti `LD\_PRELOAD`/`LD\_LIBRARY\_PATH` sensitif, tetapi secure-execution mode dapat mengabaikannya pada setuid contexts. Jangan berasumsi environment trick berlaku; validasi binary, loader behavior, dan sudoers policy.

10. Credential & Configuration Exposure

Privilege escalation kadang bukan executable abuse tetapi credential reuse: service password, backup credential, database secret, private key, token, atau admin password tersimpan terlalu longgar.

```
find /opt /srv /var/www -maxdepth 4 -type f \
  \( -name '*.env' -o -name '*.conf' -o -name '*.ini' -o -name '*.yaml' \) \
  -readable 2>/dev/null | head -n 80

grep -RiLe 'password|token|secret|api[_]?key' /opt/app 2>/dev/null | head
```

Handle secrets as evidence Jika engagement menemukan credential valid, jangan masukkan plaintext ke report/screenshot/chat. Record location, privilege, masked identifier, and impact; store raw secret only in approved evidence storage if truly needed.

11. Containers, Docker Socket & Omarchy

Container boundary adalah privilege boundary sendiri. Docker daemon biasanya root; akses ke daemon/socket dapat memberi root-level control atas host. Docker docs memperingatkan bahwa `docker` group grants root-level privileges.

```
id -nG
ls -l /var/run/docker.sock 2>/dev/null
findmnt | grep -E 'docker|container' | head -n 20
```

Omarchy secara default tidak memasukkan user ke `docker` group dan menjalankan Docker CLI via `sudo`, karena permanent group membership secara praktis adalah passwordless root. Pertahankan default ini untuk workstation security.

No breakout recipe in this module Kita tidak perlu menjalankan container-escape command untuk membuktikan risk. Membership/access ke privileged Docker socket sudah merupakan security finding yang harus dievaluasi terhadap intended admin model.

12. Kernel Exploits: Last Resort

Local kernel exploit bisa relevan ketika configuration paths tidak ada, tetapi harus diperlakukan sebagai high-risk action: build-specific, mitigation-sensitive, potentially unstable, dan dapat crash/reboot target.

```
uname -a
cat /proc/version
cat /etc/os-release
sysctl kernel.unprivileged_userns_clone 2>/dev/null || true
```

Decision gate before any kernel PoC

[] Exact kernel/build affected oleh vendor advisory?

- [] Architecture dan config match?
- [] Mitigation/backport status sudah dicek?
- [] Snapshot/restore tersedia?
- [] RoE mengizinkan kernel exploitation?
- [] Exploit code sudah diaudit?
- [] Ada lower-risk path?

Version string is not enough Distro vendor sering backport patch tanpa mengganti upstream version seperti yang diharapkan scanner. Vendor advisory/package revision lebih authoritative daripada sekadar `uname`.

13. Automation: LinPEAS After Manual Enumeration

LinPEAS adalah enumeration assistant, bukan vulnerability oracle. Jalankan setelah manual pass agar lo bisa membandingkan: apa yang lo temukan sendiri, apa yang tool tambahkan, dan mana yang false positive/context-dependent.

Safer acquisition pattern

- Gunakan official `peass-ng/PEASS-ng` release asset.
- Bandingkan SHA-256 asset dengan checksum yang dipublikasikan pada GitHub release.
- Simpan tool di `~/CyberSec/tools/` atau workspace engagement; jangan pipe remote script langsung ke shell.

```
sha256sum linpeas.sh
chmod +x linpeas.sh
./linpeas.sh > linpeas.out
less -R linpeas.out
```

Current release note Saat modul ini dibuat, PEASS-ng release page menerbitkan SHA-256 untuk assets seperti `linpeas.sh`. Check release terbaru pada hari lo belajar; jangan hard-code checksum lama sebagai trust anchor.

14. Local Training Lab Architecture

Lab sengaja memiliki beberapa misconfiguration sekaligus. Gunakan `reset` sebelum setiap scenario supaya hasil sebelumnya tidak merusak reasoning lab berikutnya.

```
Omarchy host
| sudo docker exec --user trainee
v
+-----+
| mod16-linux-privesc (network=none) |
| trainee uid=1000                   |
|                                     |
| sudo helper -> root-only flag     |
| SUID reader -> root-only flag     |
| file capability -> root-only flag |
| root cron -> writable task.sh     |
| readable app config -> lab password |
+-----+
```

Isolation trade-off Lab TIDAK memakai `--privileged`, host volume, Docker socket, atau network. `no-new-privileges` sengaja tidak digunakan karena akan mematikan SUID/file-capability exercise. Risk dikompensasi dengan disposable container + no network + no host mounts + resource limits.

Build & start

```
cd modules/16-linux-privilege-escalation/labs
./linux-privesc-lab.sh build
./linux-privesc-lab.sh create
./linux-privesc-lab.sh shell
```

15. Labs 1-4: Baseline, sudo, SUID, Capabilities

Lab 15.1 - Manual baseline

```
id
uname -a
cat /etc/os-release
sudo -l
find / -xdev -perm -4000 -type f 2>/dev/null
getcap -r / 2>/dev/null
```

Lab 15.2 - sudo policy -> privileged file read

```
sudo -l
sudo /usr/local/bin/lab-backup /root/flag-sudo.txt
cat /tmp/backup-output
```

Root cause: sudoers memberi root execution pada helper yang menerima arbitrary path dan menyalin file tanpa authorization check. Minimum proof: root-only file terbaca; tidak perlu shell root.

Lab 15.3 - custom SUID helper

```
ls -l /usr/local/bin/suid-reader
/usr/local/bin/suid-reader /root/flag-suid.txt
```

Inspect source di repo lab. Cari effective UID transition dan arbitrary path input.

Lab 15.4 - file capability

```
getcap /usr/local/bin/cap-reader
/usr/local/bin/cap-reader /root/flag-cap.txt
```

Bandingkan: process tetap bukan UID 0, tetapi capability tertentu cukup untuk melewati permission boundary yang dibutuhkan.

16. Labs 5-8: Cron, Credentials, Process & Evidence

Lab 16.1 - Observe scheduled root job

```
ps auxww | grep -E '[c]ron|maintenance'
ls -l /opt/lab-maintenance/task.sh
cat /etc/cron.d/mod16-maintenance
```

Optional: jalankan pspy bila sudah lo download/verify, lalu tunggu job root muncul.

Lab 16.2 - Writable root-executed script

```
cat > /opt/lab-maintenance/task.sh <<'EOF'
#!/bin/sh
cp /root/flag-cron.txt /tmp/cron-proof
chmod 0644 /tmp/cron-proof
EOF
chmod +x /opt/lab-maintenance/task.sh

# tunggu <= 1 menit
cat /tmp/cron-proof
```

Minimum proof lagi-lagi bukan persistent shell. Root job menjalankan user-writable code; privileged file disclosure cukup membuktikan boundary failure.

Lab 16.3 - Readable application credential

```
stat -c '%A %U:%G %n' /opt/app/config.env
cat /opt/app/config.env
su - root
# password hanya dummy credential di disposable container
```

Setelah masuk root, cukup `id` dan `cat /root/flag-cred.txt`, lalu `exit`. Jangan membuat persistence.

Lab 16.4 - Evidence

```
exit
./linux-privesc-lab.sh evidence module16-privesc/04-evidence
./evidence-manifest.sh module16-privesc/04-evidence
```

17. Reset, Cleanup & Retest

Setiap scenario harus bisa direproduksi dari keadaan bersih. Reset container menghapus state hasil lab, termasuk modified cron script dan `/tmp` artifacts.

```
./linux-privesc-lab.sh reset
./linux-privesc-lab.sh status

# selesai seluruh module
./linux-privesc-lab.sh cleanup
```

Finding template

```
python labs/finding-note.py \
  module16-privesc/03-findings/cron-writable-script.md \
  --title 'Root cron executes user-writable script' \
  --vector 'Scheduled task / file permissions' \
  --evidence 'root-owned cron entry executes /opt/lab-maintenance/task.sh; trainee can modify script' \
  --impact 'low-privilege user can influence code executed as root' \
  --remediation 'make script root-owned/non-writable; constrain directories; review scheduled jobs'
```

18. Blue Team View & Remediation

Weakness	Detection / review	Remediation
Over-broad sudo	sudo logs; sudoers review; unusual privileged command	Allow exact safe subcommands; avoid dangerous interpreters/editors; require auth where appropriate.
Unexpected SUID/SGID	Package integrity, file mode inventory	Remove bit if unnecessary; replace custom privileged helper; minimize privileged code.
Risky capabilities	`getcap` inventory; package baseline	Remove unnecessary file caps; narrow capability; redesign service boundary.
Writable root job	FIM on scripts/unit files; process ancestry	Root-own executable/config and parent dirs; least privilege service user.
Credential exposure	Secret scanning, access audit, process args review	Secret manager; restrictive permissions; rotate/revoke exposed credentials.

Docker boundary Review Docker group/socket separately: access to a rootful Docker daemon is root-equivalent. Keep users out of the group unless that admin capability is explicitly intended.

Remediation must remove the primitive "Update LinPEAS" atau "hide the flag" bukan remediation. Perbaiki trust boundary: ownership, sudo policy, capability assignment, execution identity, secret lifecycle, atau container-admin model.

19. Capstone - Unknown Local PrivEsc Path

Reset lab. Jangan buka Dockerfile/source sampai selesai. Anggap lo mendapat shell sebagai `trainee` dan objective hanya membaca satu root-only flag melalui path yang lo temukan sendiri.

```
./linux-privesc-lab.sh reset
./linux-privesc-lab.sh shell
```

Rules

- [] Manual enumeration minimal 10 menit sebelum membuka automation output.
- [] Pilih satu path saja; jangan eksploit semua.
- [] Tidak membuat persistent root shell/backdoor.
- [] Catat baseline command, hypothesis, proof, root cause, impact, remediation.
- [] Cleanup/reset setelah proof.
- [] Bandingkan hasil manual dengan LinPEAS opsional setelah challenge selesai.

Deliverable

```
module16-privesc/
├── 01-enum/manual.txt
├── 02-evidence/proof.txt
├── 03-findings/capstone.md
└── 04-evidence-manifest.sha256
```

Exit criterion Lo lulus jika bisa menjelaskan *kenapa* path bekerja, permission boundary apa yang gagal, kenapa proof cukup, telemetry apa yang mungkin terlihat defender, dan perubahan konfigurasi apa yang menutup path.

20. Compact Cheat Sheet

```
# Identity / OS
id; groups; uname -a; cat /etc/os-release

# sudo
sudo -l

# SUID / SGID
find / -xdev -perm -4000 -type f 2>/dev/null
find / -xdev -perm -2000 -type f 2>/dev/null

# Capabilities
getcap -r / 2>/dev/null

# Services / timers / cron
systemctl --type=service --state=running --no-pager
systemctl list-timers --all --no-pager
cat /etc/crontab 2>/dev/null
ls -la /etc/cron.d 2>/dev/null

# Filesystem / mounts / PATH
findmnt -o TARGET,SOURCE,FSTYPE,OPTIONS
printf '%s\n' "$PATH" | tr ':' '\n'
namei -l /interesting/path

# Processes / sockets
ps auxww --forest
ss -lntup 2>/dev/null
```

21. Knowledge Check

1. Kenapa `sudo -l` lebih berguna daripada hanya mengecek membership group sudo/wheel?
2. Apakah semua SUID binary adalah vulnerability? Jelaskan.
3. Apa beda SUID root dengan file capability?
4. Kenapa `CAP\_DAC\_READ\_SEARCH` dapat sensitif pada file-reader utility?
5. Apa yang harus dicek selain writable file ketika menganalisis service root?
6. Bagaimana pspy dapat melihat short-lived process tanpa root?
7. Kenapa secret di command-line arguments berisiko?
8. Apa hubungan writable scheduled script dengan privilege escalation?
9. Kenapa container Docker module menggunakan `--network none` dan tanpa host mounts?
10. Kenapa `docker` group dianggap root-equivalent?
11. Mengapa kernel exploit harus menjadi last resort?
12. Kenapa distro backport membuat version-only kernel matching lemah?
13. Apa nilai LinPEAS setelah manual enumeration?

14. Apa contoh minimum sufficient proof yang lebih aman daripada root shell?
15. Mengapa plaintext credential sebaiknya tidak masuk report?
16. Apa yang dimaksud memperbaiki privilege primitive, bukan symptom?

22. References & Next Step

- Linux capabilities(7): <https://man7.org/linux/man-pages/man7/capabilities.7.html>
- sudo / sudoers documentation: <https://www.sudo.ws/docs/man/>
- systemd manuals: <https://www.freedesktop.org/software/systemd/man/latest/>
- Docker Linux post-install security note: <https://docs.docker.com/engine/install/linux-postinstall/>
- Omarchy Development Tools / Docker security: <https://omarchy.org/manual/development-tools/>
- GTFOBins: <https://gtfobins.github.io/>
- PEASS-ng / linPEAS: <https://github.com/peass-ng/PEASS-ng>
- pspy: <https://github.com/DominicBreuker/pspy>

Exit checklist before Module 17

- [] Gue bisa menjelaskan effective UID/GID, SUID/SGID, sudo rule, dan Linux capability tanpa menghafal tool output.
- [] Gue selalu mulai manual enumeration sebelum automation.
- [] Gue bisa membedakan writable file biasa vs file yang benar-benar berada di privileged execution path.
- [] Gue bisa memberikan minimum sufficient proof tanpa membuat persistence.
- [] Gue tahu kenapa Docker socket/group adalah root-equivalent boundary pada host.
- [] Gue bisa menyebutkan telemetry dan remediation untuk path yang gue temukan.
- [] Gue selalu reset/cleanup disposable lab dan tidak menyimpan plaintext lab credential di repo.

Repo package contents

```
modules/16-linux-privilege-escalation/
├── README.md
├── docx/
├── pdf/
├── labs/
│   ├── workspace-init.sh
│   ├── linux-privesc-lab.sh
│   ├── finding-note.py
│   ├── evidence-manifest.sh
│   └── linux-privesc-lab/
│       ├── Dockerfile
│       ├── suid_reader.c
│       ├── cap_reader.c
│       ├── lab-backup
│       ├── task.sh
│       └── entrypoint.sh
```

Next module Module 17 - Windows Privilege Escalation: tokens/privileges, services, scheduled tasks, ACLs, registry, credential artifacts, UAC concepts, WinPEAS/Sysinternals, and a disposable Windows training workflow.